Post mortem · Alfa (v0.1-alpha)

Myrmutation · Gutix (GutixStudio) · Juegos para Web y Redes Sociales · URJC 2025-26
Entrega: 11 de octubre de 2026

1. Trabajo colectivo

Lo que ha funcionado

- **Base común desde el primer día.** La arquitectura compartida (EventBus, GameManager, ResourceManager, salas y datos en ScriptableObjects) permitió que cada uno programase su sistema sabiendo cómo iba a encajar con los demás, sin tocar scripts ajenos.
- **Flujo de Git.** Una rama por tarea, nada en `develop` sin pull request revisada, todo desde Git Bash y GitHub.
- **Escenas de prueba por persona.** Cada sistema se probó en su propia escena, así que casi no nos hemos pisado.
- **Tablero y comunicación.** Tareas con responsable, estimación y dependencias en GitHub Projects, y coordinación por Teams.
- **Documentación técnica.** La IA y la genética quedaron documentadas en el repositorio, lo que ahorró preguntas entre compañeros.

Lo que no ha funcionado tan bien

- **Dependencias entre tareas.** Muchas tareas dependían de las de otros: el cerebro de las hormigas necesitaba las salas y la producción; la cría, la genética y las estaciones; y la integración final, todo lo anterior. Esto ha provocado esperas.
- **Integración concentrada al final.** Al juntar los sistemas en `Game.unity` aparecieron errores que no se veían en las escenas de prueba por separado.
- **Conflictos con Git y Unity.** Sobre todo con los archivos `.meta` (unos no se subían y otros chocaban) y con dos escenas de prueba que se llamaban igual. Se resolvieron sin perder trabajo de nadie.
- **Arte después del código.** Durante buena parte del desarrollo el juego ha funcionado con imágenes provisionales.

Lecciones aprendidas

1. **Integrar poco a poco.** En la beta, fusionar en `Game.unity` de forma progresiva durante el sprint, no todo al final.
2. **Revisar los `.meta` en cada commit.** Antes de una PR, comprobar que cada archivo nuevo va con su `.meta` y que no se suben `.meta` huérfanos.
3. **Nombrar las escenas de prueba con el ID de la tarea** (`Test_D1`, `Test_D3`…) para que no se repitan.

4. **Fijar antes los requisitos técnicos del arte** (tamaño en píxeles, PPU y proporciones) para que las ilustraciones encajen a la primera.
5. **Planificar las dependencias** en el tablero para que nadie se quede bloqueado esperando a otro.

2. Trabajo individual

Ayoze González — Producción, UI y web (E) · Arquitectura e integración (A)

En lo que respecta a mi trabajo, me he encargado de dos áreas: la arquitectura e integración del proyecto en Unity (A) y la producción, la interfaz y la parte web (E). Considero que haber dejado desde el primer día una base común de contratos (EventBus, GameManager, ResourceManager y los datos en ScriptableObjects), junto con un tablero de tareas con responsables, estimaciones y dependencias, ha sido clave para que seis personas pudieran trabajar en paralelo sin pisarse. El flujo de ramas main, develop y feature, con pull requests revisadas y escenas de prueba separadas para cada sistema, ha permitido llegar a la entrega con un producto que cumple las especificaciones de la alfa.

En cuanto a los contras, al ocuparme de la integración final (A5), mi trabajo dependía de que todas las demás tareas estuvieran terminadas y fusionadas, lo que ha concentrado gran parte de la carga en los últimos días. Además, al juntar los sistemas en la escena principal aparecieron problemas que no se veían en las escenas de prueba por separado: salas que empezaban a construirse antes de que llegaran las obreras, hormigas superpuestas que no se podían seleccionar, un componente duplicado que alimentaba dos larvas a la vez o presas de expedición que no había forma de comerse en la partida. Resolverlos ha llevado más tiempo del previsto, y he contado con la ayuda de Naroa en la integración y en las pruebas. Igual que mis compañeros, no lo considero un fallo de organización, sino algo propio de ser la tarea que cierra el proyecto. De cara a la beta, propondría integrar de forma progresiva a lo largo del sprint en lugar de dejarlo todo para el final.

Uno de los aspectos más destacados ha sido conseguir que todos los sistemas (IA de las hormigas, construcción, economía, cría, genética y expediciones) funcionen juntos en una partida completa, además de ajustar las reglas de construcción para que una sala solo avance con el mínimo de obreras presentes y organizar el HUD para que ningún panel se solape. Por la parte de producción, he coordinado el reparto de tareas, redactado el GDD y preparado las pantallas obligatorias, las cuentas del estudio y la publicación en itch.io.

Las horas dedicadas a cada tarea de mi campo han sido las siguientes:

Horas para

- **A1 (integrar arquitectura base): 3** Subir la arquitectura común para que todo el equipo pudiera trabajar en paralelo.

- **A2 (escena Game y cámara): 2** Escena principal con el hormiguero y una cámara con arrastre y zoom en PC y móvil.
- **A3 (navegación por túneles): 2** Grafo de waypoints para que las hormigas vayan de una sala a otra.
- **A4 (construcción simplificada): 2** Construir salas en las zonas prediseñadas según las obreras asignadas.
- **A5 (integración en Game.unity): 4** Juntar todos los sistemas para poder jugar una partida completa.
- **E1 (reunión y reparto): 1** Arrancar el sprint y asignar roles.
- **E2 (cargar el tablero): 1** Crear las issues con responsable, estimación y fecha.
- **E3 (pantallas obligatorias): 2** Menú principal, Créditos y Game Over con navegación entre escenas.
- **E4 (cuentas y enlaces): 1** Redes del estudio y archivo de enlaces.
- **E5 (HUD de partida): 1** Recursos, día y estación, velocidades y paneles de Guardería y Expedición.
- **E6 (build en itch.io): 1** Versión jugable publicada y probada en navegador.
- **E7 (GDD): 4** Documento de diseño completo con control de versiones.

Íñigo García — Inteligencia artificial (B)

La verdad es que lo que mejor nos ha funcionado ha sido organizarnos bien desde el principio. Cada tarea iba en su propia rama de GitHub (todo desde Git Bash) y nada se metía en develop sin pasar antes por una pull request. Además, cada uno probaba sus cosas en su propia escena dentro del proyecto de Unity, así que casi no nos hemos pisado entre nosotros. Gracias a eso la alfa ha salido como se pedía. Yo además dejé apuntado en el repo cómo conectar cualquier cosa con las hormigas, para que mis compañeros no tuvieran que venir a preguntarme cada dos por tres.

Eso sí, no todo ha sido perfecto. Repartirse el trabajo está genial porque avanzamos todos a la vez, pero muchas cosas dependen de otras aunque las haga otra persona. Por ejemplo, el cerebro de las hormigas no pintaba mucho hasta que no estuvieron hechas las salas y la producción. Y el panel de selección ya está terminado, pero no se va a ver en la escena principal hasta que el encargado de esa escena lo ponga. También nos hemos peleado un poco con Git y Unity, sobre todo con los dichosos archivos .meta: unos no se subían y otros chocaban entre compañeros. Pero vamos, no lo veo como un fallo del proyecto, es lo normal cuando haces un juego entre varios.

Lo mío ha sido la inteligencia artificial de las hormigas. En este juego no les dices qué hacer paso a paso, sino que van a su aire, así que es bastante importante que se comporten de forma que tenga sentido. Ha tenido su miga, pero como es una alfa lo importante era que funcionara, y lo he podido sacar sin pasarme de los tiempos de cada tarea. De lo que más contento estoy es de haber separado la IA en tres partes (lo que la hormiga nota, lo que decide y lo que hace), y de que cada una decida sola si se pone a trabajar en su sala, se va a la despensa a comer o se para a descansar, siempre mirando primero lo que necesita. También mola que los genes de mis compañeros se noten en cómo se comportan: por ejemplo, una hormiga con la tara Glotona se zampa el doble de comida cada vez que come.

Más o menos, las horas que le he echado a cada tarea han sido estas:

Horas para

- **B1 (prefab de la hormiga): 3** Montar la hormiga con el cuerpo por partes y hacer que pueda moverse, pararse, trabajar y comer, y que sepa si tiene hambre, si está cansada o cuál es la sala más cercana.
- **B2 (cerebro de la hormiga): 1** Que cada hormiga decida sola qué hacer, poniendo por delante lo que necesita antes que su trabajo.
- **B3 (necesidades: hambre y energía): 1** Que el hambre y la energía cambien con el tiempo y según lo que esté haciendo, y que pueda agotarse o morirse de hambre.
- **B4 (selección y panel de hormiga): 2** Poder pinchar en una hormiga, ver sus datos y mandarla a una sala, dejando hueco para que otros añadan sus botones, como el de Devorar.

Alonso Cancela — Genética (C)

En lo que respecta al trabajo que se ha realizado por mi parte, he podido apreciar que gracias a la organización que se ha mantenido desde el principio, se ha conseguido un producto conforme a las especificaciones del proyecto. Tanto el haber empleado la plataforma de Git Bash junto a GitHub, como el haber separado las tareas de cada uno en escenas diferentes del mismo proyecto de Unity, han contribuido a ello.

En cuanto a los contras del proyecto, si bien es cierto que la separación de tareas ha ayudado a agilizar el trabajo, el hecho de que unas tareas dependieran de otras (sin necesariamente ser del mismo miembro) ha dado alguna ocasión en donde haya que esperar a que un compañero termine una tarea para poder tú hacer la pendiente. Este caso no es realmente un contra del proyecto, sino más bien un retraso entendible por la naturaleza de este en cuanto a su creación.

Por mi parte, me he dedicado al campo de la genética de las hormigas, una de las mecánicas novedosas de esta experiencia. Si bien no ha sido un reto sencillo, al ser una alfa en donde la prioridad ha sido que funcione, se ha conseguido completar sin retrasar los tiempos esperados de cada tarea. Uno de los aspectos más destacados a la hora de realizarlo ha sido el lograr que, dependiendo de la casta y los genes originales de cada hormiga, se pueda formar una mutación que afecte únicamente a la descendencia.

Las horas a las que se les ha dedicado tiempo a cada tarea del campo asignado han sido las siguientes:

Horas para

- **C1 (datos de genes de la alfa): 2** Crear los genes como datos editables.
- **C2 (genoma de la hormiga): 2** Que los genes cambien de verdad los stats.
- **C3 (devorar, Festín): 1 y media** La mecánica clave: una hormiga se come a otra y muta.
- **C4 (herencia por la reina): 1 y media** Que la mutación pase a las siguientes generaciones.

- **C5 (feedback visual de mutación): 1 y media** Que se note cuando una hormiga muta.

Ignacio Perea — Economía y cría (D1–D2)

En esta alfa me he encargado sobre todo de la parte de economía de la colonia y de que la colonia pueda crecer, que en el tablero eran las tareas D1 (producción de recursos) y D2 (reina y cría).

En la D1 lo que buscaba era que los recursos tuvieran una lógica detrás. Las obreras que trabajan en la sala de hongos van gastando hojas y con el tiempo sacan hongo, y luego ese hongo se acaba convirtiendo en comida (se puede hacer directamente o pasando por la despensa, lo dejé con las dos opciones). Lo que más me gusta es que no da igual quién trabaje: cuantas más obreras haya más se produce, pero también cuenta la casta de cada una, así que una “Menor” rinde bastante mejor en los hongos que una “Mayor”. Si se acaban las hojas la sala simplemente se queda esperando, no pierde lo que llevaba hecho.

La D2 es la que hace que la colonia vaya creciendo. La reina pone un huevo cada x tiempo siempre que haya comida y hueco en la guardería, y de ahí pasa a larva, luego a pupa y al final nace la hormiga. La parte que me parece más interesante es que la casta no viene por los genes sino por cómo alimentas a la larva, igual que en las hormigas de verdad. Puedes hacerla Menor, Media o Mayor, y cada una cuesta una cantidad distinta de comida y sale con su tamaño, su color y sus stats. Encima al nacer hereda los genes que tenga la reina guardados para esa casta. Todos los números se pueden cambiar desde el inspector para poder equilibrarlo todo más adelante sin tocar código.

En cuanto al trabajo en equipo, lo que más me ha ayudado es que la base del proyecto (recursos, salas, eventos, los datos de las castas...) estuviera hecha desde el principio, porque así podía programar lo mío sabiendo cómo iba a encajar con lo de los demás y sin tener que meterme en sus scripts. Lo de tener cada tarea en su rama y que todo pasara por pull request también ha estado bien. Lo malo es que mis tareas dependían bastante de las de otros: para la producción hacían falta las salas y las hormigas, y para la cría la genética y las estaciones. También hemos tenido algún que otro conflicto al juntar cambios, sobre todo con los .meta y con que acabamos teniendo dos escenas de prueba que se llamaban igual mi compañero Nicolás y yo. Al final las juntamos en una sin perder nada de nadie, así que tampoco fue para tanto, y por lo menos ya sabemos qué no hacer la próxima vez.

Tiempo dedicado:

- **D1 (producción de recursos): 2:30 h** Que las hojas se conviertan en hongo y el hongo en comida, dependiendo de las obreras que haya y de su casta.
- **D2 (reina y cría): 2 h** Que la reina ponga huevos, el ciclo de huevo a adulta, elegir la casta al dar de comer a la larva y que la cría herede los genes.

Nicolás Rosón — Estaciones y expediciones (D3–D4) · Portfolio

Mi trabajo en esta alfa se ha centrado en dos sistemas de la colonia, las estaciones (D3) y las expediciones (D4), y parte del portfolio del estudio en GitHub Pages.

Con D3 quería que el invierno se notara de verdad. Al llegar, cada hormiga pide una ración diaria de comida, el hambre sube más rápido y no se puede salir del nido. Si la despensa no da para todas, caen obreras, y la reina es la última en caer. Si cae ella, la partida termina. Si la colonia llega viva al final del invierno, se gana. Todos los valores se pueden tocar desde el inspector, así que equilibrarlos más adelante será rápido.

D4 permite mandar de una a tres obreras fuera durante un rato. Vuelven con hojas o, con algo de suerte, con una presa, que es lo que luego alimenta la mutación en el Festín. Cuantas más obreras salen, más probable es encontrar presa, y las que cargan más traen más hojas.

Del trabajo en equipo valoro sobre todo que la base común del proyecto estuviera lista antes de empezar. Gracias a eso pude programar sabiendo cómo iba a hablar con los sistemas de los demás. También valoro mucho la comunicación y coordinación que hemos tenido entre todos los miembros para organizarnos de la mejor manera posible y repartirnos nuestras tareas adecuadamente. La parte negativa es que mis tareas tocan muchas piezas ajenas y tuve que ajustar los tiempos con compañeros para poder coordinarnos correctamente y trabajar en nuestras tareas sin perjudicar a ningún otro miembro. Ha costado algún rato de espera y algún conflicto al mezclar cambios, aunque nada grave.

Tiempo dedicado:

- **D3 (estaciones y fin de partida): 3 h** Reglas del invierno, victoria y derrota.
- **D4 (expedición mínima): 2:30 h** Salidas al exterior con botín de hojas o presas.
- **E10 (portfolio en GitHub Pages): 2:30 h** Web del estudio con el equipo, el juego y las redes.

Naroa Jiménez — Arte (F) · Apoyo en integración y pruebas

En lo que respecta a mi trabajo, me he encargado del apartado artístico del juego. Gracias a la organización del equipo desde el principio, con un listado de piezas que indicaba para qué servía cada una, qué tamaño debía tener y en qué orden hacerlas, he podido centrarme en dibujar sin tener que adivinar qué necesitaba cada sistema. Haber definido antes una paleta y un estilo común ha ayudado a que las salas tengan coherencia entre sí y con la temática del juego.

Además del arte, he ayudado a Ayoze en la integración de los sistemas en la escena principal (Game) y he realizado varias pruebas del juego: comprobar que las nuevas hormigas heredan las mutaciones de la reina, que la construcción de salas respeta el mínimo de obreras y que la partida se puede jugar de principio a fin. Esto me ha servido para entender cómo encajan mis ilustraciones en el juego y para detectar errores que no se veían en las escenas de prueba de cada sistema.

En cuanto a los contras, el arte depende de que las mecánicas estén definidas, y a la vez el código necesita las imágenes para verse bien. Por eso, durante buena parte del desarrollo el juego ha funcionado con imágenes provisionales y mis piezas se han ido incorporando a medida que las terminaba. También ha habido que ajustar las ilustraciones a requisitos técnicos, como el tamaño en píxeles y la proporción de cada sala, para que encajaran en las zonas del hormiguero sin deformarse. Igual que mis

compañeros, no lo considero un fallo del proyecto, sino algo propio de trabajar en paralelo en tan poco tiempo.

Uno de los aspectos más destacados ha sido conseguir que cada sala se distinga de un vistazo (Real, Guardería, Despensa, Hongos y Festín), ya que es lo que más ve el jugador durante la partida. De cara a la beta, quedan pendientes el fondo con los estratos del hormiguero y la sustitución de la interfaz provisional por una con el estilo propio del juego.

Las horas dedicadas a cada tarea de mi campo han sido las siguientes:

Horas para

- **F1 (paleta y guía de estilo): 4** Fijar el estilo visual antes de producir.
- **F3 (escenario y salas): 8** Las ilustraciones de las salas del hormiguero.
- **Apoyo en la integración (Game): 4** Ayudar a juntar los sistemas en la escena principal.
- **Pruebas del juego: 3** Comprobar que las mecánicas funcionan juntas y detectar errores.

3. Resumen de horas

Miembro	Área	Horas
Ayoze González	A1–A5 · E1–E7	24 h
Íñigo García	B1–B4	7 h
Alonso Cancela	C1–C5	8,5 h
Ignacio Perea	D1–D2	4,5 h
Nicolás Rosón	D3–D4 · E10	8 h
Naroa Jiménez	F · apoyo en integración y pruebas	19 h